



Query Parsing





Query Parsing



Query Parsing

**Regular
Expressions**

Query Parsing



Queries are **hard coded** using operator framework

Query 1: "{1} WHERE {1} > 2 AND {1} < 6"

Query 2: "SUM{1} WHERE {1} > 2 AND {1} < 6"

Query 3: "SUM{1} GROUP BY {2} WHERE {1} > 2 AND {1} < 6"

Query Parsing

```
QueryComponents parseQuery(const std::string &query) {  
    QueryComponents components;  
    // Example: Parse SELECT attributes  
    std::regex selectRegex("\\{((\\d+)\\)}, \\{((\\d+)\\)}?");  
    std::smatch selectMatches;  
    std::string::const_iterator queryStart(query.cbegin());  
    while (std::regex_search(queryStart, query.cend(), selectMatches, selectRegex)) {  
        ...  
    }  
    // Further parsing for SUM, GROUP BY, and WHERE conditions  
    return components;  
}
```


Query Parsing

Regex

```
QueryComponents parseQuery(const std::string &query) {  
    QueryComponents components;  
    // Example: Parse SELECT attributes  
    std::regex selectRegex("\\{((\\d+)\\)}, \\{((\\d+)\\)}?");  
    std::smatch selectMatches;  
    std::string::const_iterator queryStart(query.cbegin());  
    while (std::regex_search(queryStart, query.cend(), selectMatches, selectRegex)) {  
        ...  
    }  
    // Further parsing for SUM, GROUP BY, and WHERE conditions  
    return components;  
}
```

Query Parsing

Regex

Query
Parsing

```
QueryComponents parseQuery(const std::string &query) {  
    QueryComponents components;  
    // Example: Parse SELECT attributes  
    std::regex selectRegex("\\{((\\d+)\\)}, \\{((\\d+)\\)}?");  
    std::smatch selectMatches;  
    std::string::const_iterator queryStart(query.cbegin());  
    while (std::regex_search(queryStart, query.cend(), selectMatches, selectRegex)) {  
        ...  
    }  
    // Further parsing for SUM, GROUP BY, and WHERE conditions  
    return components;  
}
```


Query Components

```
struct QueryComponents {  
    std::vector<int> selectAttributes;  
    bool sumOperation = false;  
    int sumAttributeIndex = -1;  
    bool groupBy = false;  
    int groupByAttributeIndex = -1;  
    ...  
};
```

Query Components

```
struct QueryComponents {  
    std::vector<int> selectAttributes;  
    bool sumOperation = false;  
    int sumAttributeIndex = -1;  
    bool groupBy = false;  
    int groupByAttributeIndex = -1;  
    ...  
};
```


Query Components

COMPONENT	TYPE	DESCRIPTION
selectAttributes	std::vector<int>	Indices of attributes selected in the query.
sumOperation	bool	Indicates whether a SUM operation is included.
sumAttributeIndex	int	Targets the attribute for the SUM operation.
groupBy	bool	Specifies if there's a GROUP BY clause.
groupByAttributeIndex	int	Determines which attribute to group by.
whereCondition	bool	Checks for the presence of a WHERE clause with bounds.
whereAttributeIndex	int	Determines which attribute to filter by.
lowerBound	int	Specifies the lower bound for the filtered column
upperBound	int	Specifies the upper bound for the filtered column

Query Components

COMPONENT	TYPE	DESCRIPTION
selectAttributes	std::vector<int>	Indices of attributes selected in the query.
sumOperation	bool	Indicates whether a SUM operation is included.
sumAttributeIndex	int	Targets the attribute for the SUM operation.
groupBy	bool	Specifies if there's a GROUP BY clause.
groupByAttributeIndex	int	Determines which attribute to group by.
whereCondition	bool	Checks for the presence of a WHERE clause with bounds.
whereAttributeIndex	int	Determines which attribute to filter by.
lowerBound	int	Specifies the lower bound for the filtered column
upperBound	int	Specifies the upper bound for the filtered column

Query Components

COMPONENT	TYPE	DESCRIPTION
selectAttributes	std::vector<int>	Indices of attributes selected in the query.
sumOperation	bool	Indicates whether a SUM operation is included.
sumAttributeIndex	int	Targets the attribute for the SUM operation.
groupBy	bool	Specifies if there's a GROUP BY clause.
groupByAttributeIndex	int	Determines which attribute to group by.
whereCondition	bool	Checks for the presence of a WHERE clause with bounds.
whereAttributeIndex	int	Determines which attribute to filter by.
lowerBound	int	Specifies the lower bound for the filtered column
upperBound	int	Specifies the upper bound for the filtered column

Query Components

COMPONENT	TYPE	DESCRIPTION
selectAttributes	std::vector<int>	Indices of attributes selected in the query.
sumOperation	bool	Indicates whether a SUM operation is included.
sumAttributeIndex	int	Targets the attribute for the SUM operation.
groupBy	bool	Specifies if there's a GROUP BY clause.
groupByAttributeIndex	int	Determines which attribute to group by.
whereCondition	bool	Checks for the presence of a WHERE clause with bounds.
whereAttributeIndex	int	Determines which attribute to filter by.
lowerBound	int	Specifies the lower bound for the filtered column
upperBound	int	Specifies the upper bound for the filtered column

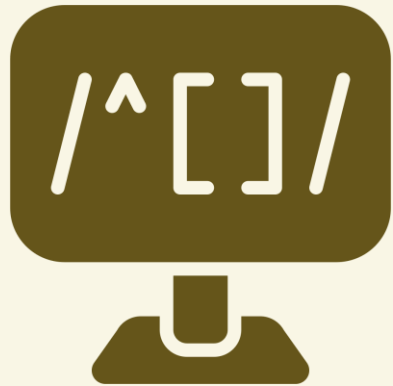
Query Components

COMPONENT	TYPE	DESCRIPTION
selectAttributes	std::vector<int>	Indices of attributes selected in the query.
sumOperation	bool	Indicates whether a SUM operation is included.
sumAttributeIndex	int	Targets the attribute for the SUM operation.
groupBy	bool	Specifies if there's a GROUP BY clause.
groupByAttributeIndex	int	Determines which attribute to group by.
whereCondition	bool	Checks for the presence of a WHERE clause with bounds.
whereAttributeIndex	int	Determines which attribute to filter by.
lowerBound	int	Specifies the lower bound for the filtered column
upperBound	int	Specifies the upper bound for the filtered column

Query Components

COMPONENT	TYPE	DESCRIPTION
selectAttributes	std::vector<int>	Indices of attributes selected in the query.
sumOperation	bool	Indicates whether a SUM operation is included.
sumAttributeIndex	int	Targets the attribute for the SUM operation.
groupBy	bool	Specifies if there's a GROUP BY clause.
groupByAttributeIndex	int	Determines which attribute to group by.
whereCondition	bool	Checks for the presence of a WHERE clause with bounds.
whereAttributeIndex	int	Determines which attribute to filter by.
lowerBound	int	Specifies the lower bound for the filtered column
upperBound	int	Specifies the upper bound for the filtered column

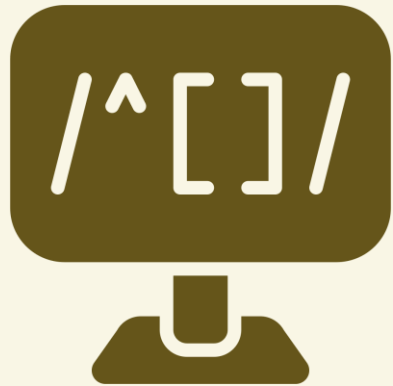
Query Parsing



**Regex
Expressions**

REGULAR EXPRESSIONS USED

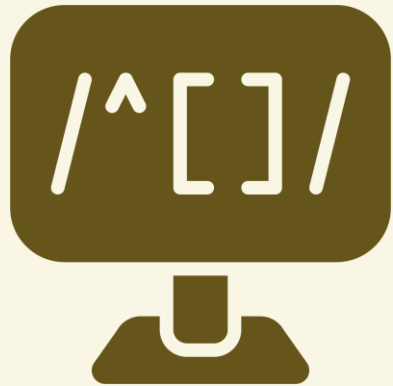
Query Parsing



**Regex
Expressions**

REGULAR EXPRESSIONS USED
Select Parsing

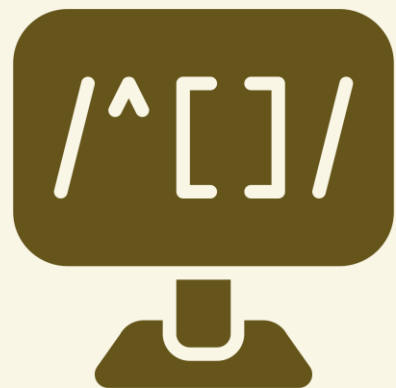
Query Parsing



Regex Expressions

REGULAR EXPRESSIONS USED
Select Parsing
SUM Detection

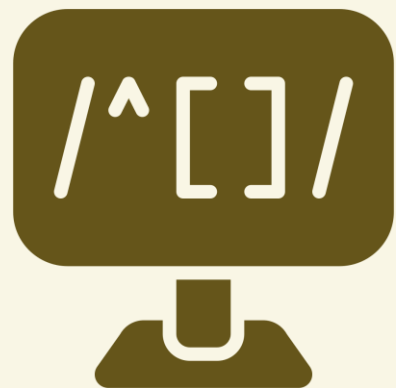
Query Parsing



Regex Expressions

REGULAR EXPRESSIONS USED
Select Parsing
SUM Detection
GROUP BY Identification

Query Parsing



Regex Expressions

REGULAR EXPRESSIONS USED
Select Parsing
SUM Detection
GROUP BY Identification
WHERE Condition Parsing

Regular Expression

REGULAR EXPRESSION	QUERY STRING
SUM\{(\d+)\}	SUM{3}

Regular Expression

REGULAR EXPRESSION	QUERY STRING
SUM\{(\d+)\}	SUM{3}

SUM
Detects &
Parses
SUM
Operation

Regular Expression

REGULAR EXPRESSION	QUERY STRING
<code>SUM\{(\d+)\}</code>	<code>SUM{3}</code>

SUM
Detects &
Parses
SUM
Operation

SUM
Matches
Text
"SUM"

Regular Expression

REGULAR EXPRESSION	QUERY STRING
SUM\{(\d+)\}	SUM{3}

SUM
Detects &
Parses
SUM
Operation

SUM
Matches
Text
"SUM"

d+
Capturing
Group
Matches
1 or More
Digits

Pretty Printing Parsed Query

```
void prettyPrint(const QueryComponents &components) {  
    std::cout << "Query Components:\n";  
    std::cout << "  Selected Attributes: ";  
    for (auto attr : components.selectAttributes) {  
        std::cout << "{" << attr + 1 << "} ";  
    }  
    ...  
}
```


Pretty Printing Parsed Query

```
void prettyPrint(const QueryComponents &components) {  
    std::cout << "Query Components:\n";  
    std::cout << "  Selected Attributes: ";  
    for (auto attr : components.selectAttributes) {  
        std::cout << "{" << attr + 1 << "} ";  
    }  
    ...  
}
```

executeQuery

```
void executeQuery(const QueryComponents &components,
                  BufferManager &buffer_manager) {
    ScanOperator scanOp(buffer_manager);
    Operator *rootOp = &scanOp; // Start with the basic scan operation
    std::optional<SelectOperator> selectOpBuffer;
    std::optional<HashAggregationOperator> hashAggOpBuffer;
    // Apply WHERE conditions
    if (components.whereAttributeIndex != -1) {
        // Construct predicates and a complex predicate for AND conditions
        selectOpBuffer.emplace(*rootOp, std::move(complexPredicate));
        rootOp = &*selectOpBuffer; // Chain the select operator
    }
    // Execute the query
}
```


executeQuery

```
void executeQuery(const QueryComponents &components,
                  BufferManager &buffer_manager) {
    ScanOperator scanOp(buffer_manager);
    Operator *rootOp = &scanOp; // Start with the basic scan operation
    std::optional<SelectOperator> selectOpBuffer;
    std::optional<HashAggregationOperator> hashAggOpBuffer;
    // Apply WHERE conditions
    if (components.whereAttributeIndex != -1) {
        // Construct predicates and a complex predicate for AND conditions
        selectOpBuffer.emplace(*rootOp, std::move(complexPredicate));
        rootOp = &*selectOpBuffer; // Chain the select operator
    }
    // Execute the query
}
```


executeQuery

```
void executeQuery(const QueryComponents &components,
                  BufferManager &buffer_manager) {
    ScanOperator scanOp(buffer_manager);
    Operator *rootOp = &scanOp; // Start with the basic scan operation
    std::optional<SelectOperator> selectOpBuffer;
    std::optional<HashAggregationOperator> hashAggOpBuffer;
    // Apply WHERE conditions
    if (components.whereAttributeIndex != -1) {
        // Construct predicates and a complex predicate for AND conditions
        selectOpBuffer.emplace(*rootOp, std::move(complexPredicate));
        rootOp = &*selectOpBuffer; // Chain the select operator
    }
    // Execute the query
}
```

WHERE clause

```
if (components.whereAttributeIndex != -1) {  
    auto complexPredicate = makeComplexPredicate(components);  
    selectOpBuffer.emplace(*rootOp, std::move(complexPredicate));  
    rootOp = &*selectOpBuffer;  
}
```


WHERE clause

```
if (components.whereAttributeIndex != -1) {  
    auto complexPredicate = makeComplexPredicate(components);  
    selectOpBuffer.emplace(*rootOp, std::move(complexPredicate));  
    rootOp = &*selectOpBuffer;  
}
```


Grouping and Aggregation

```
if (components.sumOperation || components.groupBy) {  
    std::vector<AggrFunc> aggrFuncs = prepareAggregationFunctions(components);  
    hashAggOpBuffer.emplace(*rootOp, groupByAttrs, aggrFuncs);  
    rootOp = &*hashAggOpBuffer;  
}
```

Grouping and Aggregation

```
if (components.sumOperation || components.groupBy) {  
    std::vector<AggrFunc> aggrFuncs = prepareAggregationFunctions(components);  
    hashAggOpBuffer.emplace(*rootOp, groupByAttrs, aggrFuncs);  
    rootOp = &*hashAggOpBuffer;  
}
```


Final Query Execution

```
rootOp->open();  
while (rootOp->next()) {  
    const auto &output = rootOp->getOutput();  
    printTupleFields(output);  
}  
rootOp->close();
```


Final Query Execution

```
rootOp->open();  
while (rootOp->next()) {  
    const auto &output = rootOp->getOutput();  
    printTupleFields(output);  
}  
rootOp->close();
```

Example

"SUM{1} GROUP BY {2} WHERE {1} > 2 and {1} < 6"

Hash Aggregation Operator
SUM column 1 and GROUP based on column 2

Select Operator
Select based on column 1, lower bound = 2 and upper bound = 6

Scan Operator
SCAN all columns from table using buffer manager

Example

"SUM{1} GROUP BY {2} WHERE {1} > 2 and {1} < 6"

Hash Aggregation Operator
SUM column 1 and GROUP based on column 2

Select Operator
Select based on column 1, lower bound = 2 and upper bound = 6

Scan Operator
SCAN all columns from table using buffer manager

Example

"SUM{1} GROUP BY {2} WHERE {1} > 2 and {1} < 6"

Hash Aggregation Operator
SUM column 1 and GROUP based on column 2

Select Operator
Select based on column 1, lower bound = 2 and upper bound = 6

Scan Operator
SCAN all columns from table using buffer manager

Example

"SUM{1} GROUP BY {2} WHERE {1} > 2 and {1} < 6"

Hash Aggregation Operator
SUM column 1 and GROUP based on column 2

Select Operator
Select based on column 1, lower bound = 2 and upper bound = 6

Scan Operator
SCAN all columns from table using buffer manager



Query Parsing



Query Parsing

**Regular
Expressions**

Query Compilation





Query Interpretation



**Query
Interpretation**

**Query
Compilation**

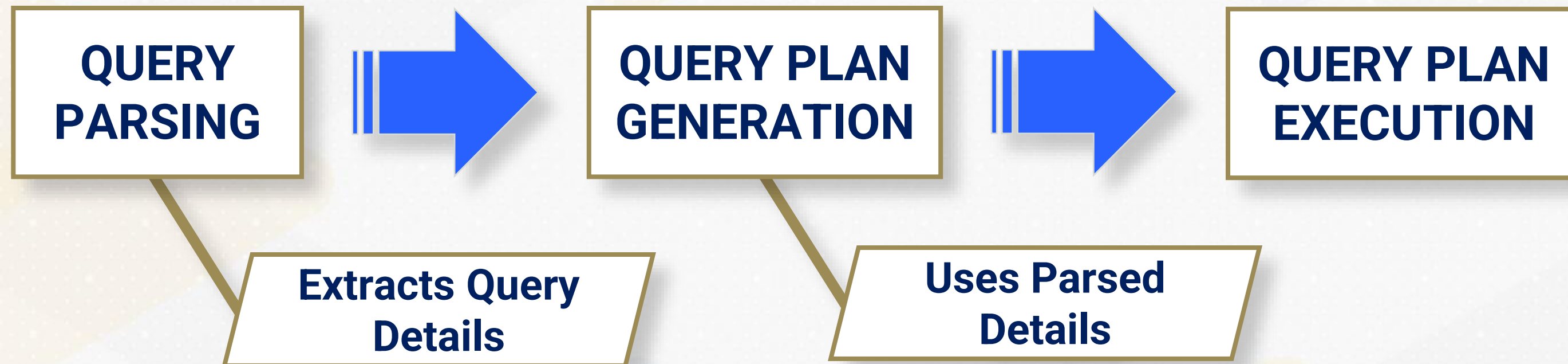
Query Interpretation



Query Interpretation



Query Interpretation



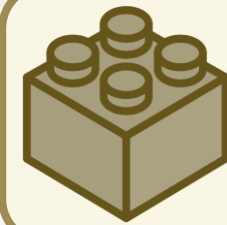
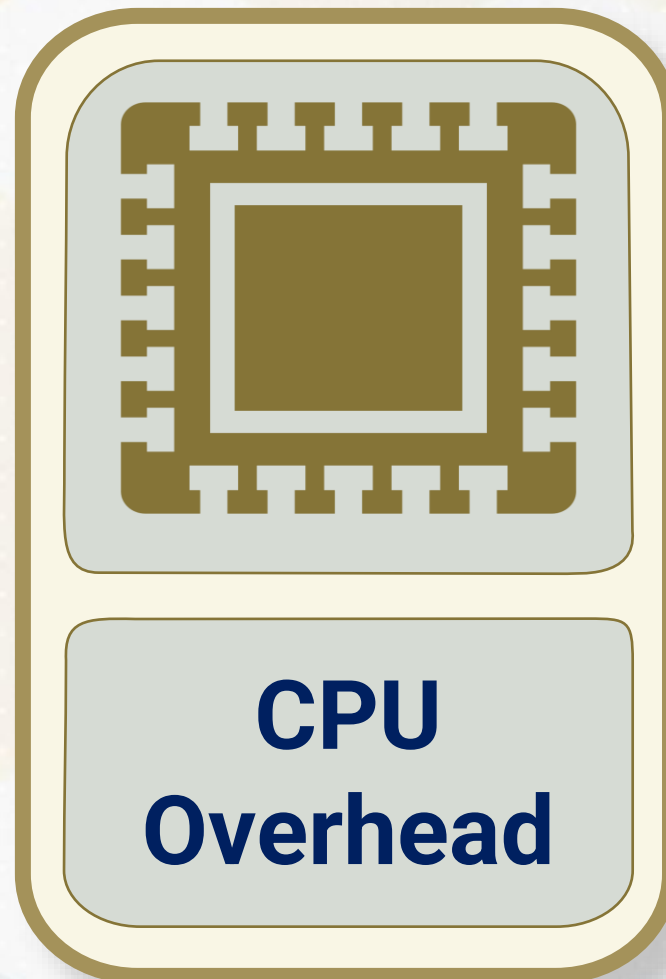
Query Interpretation



Query Interpretation

```
void queryInterpretation() {  
    std::vector<std::string> test_queries = {  
        "SUM{1} GROUP BY {1} WHERE {1} > 2 and {1} < 6"};  
    for (const auto &query : test_queries) {  
        auto components = parseQuery(query);  
        executeQuery(components, buffer_manager);  
    }  
}
```


Query Interpretation



Operator Framework



Tuple Deserialization



Polymorphism
(Operator* rootOp)



Compute Overhead

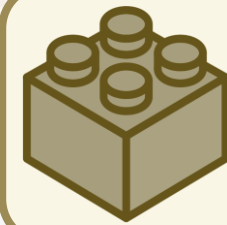
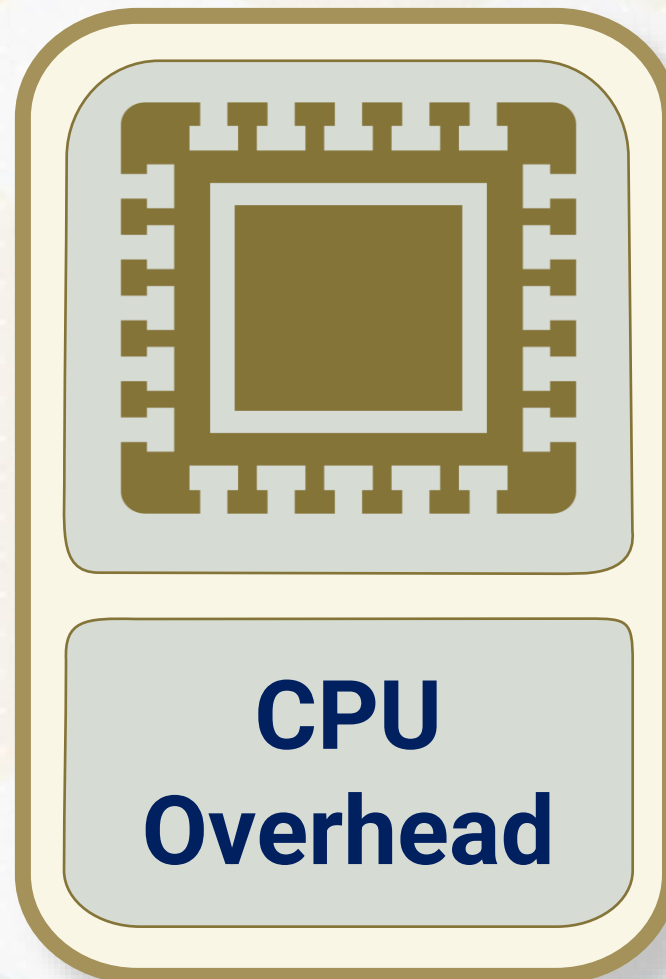


Memory Overhead



Temporary Objects
`std::unique_ptr<Field>`

Query Interpretation



Operator Framework



Tuple Deserialization



Polymorphism
(Operator* rootOp)



Compute Overhead

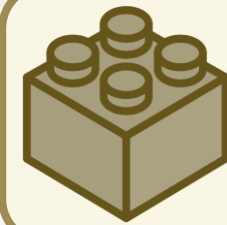
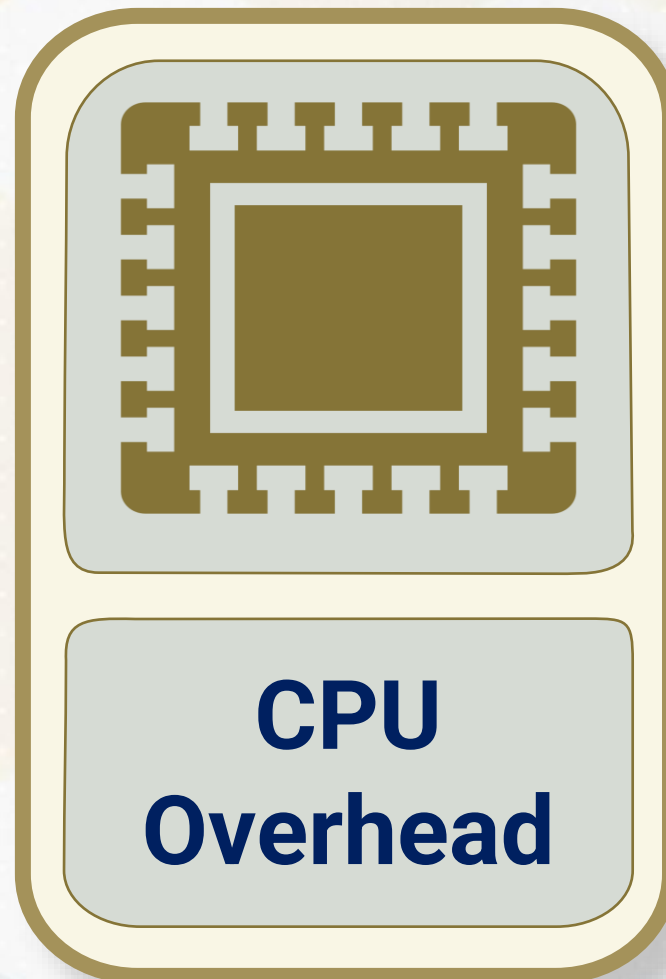


Memory Overhead



Temporary Objects
`std::unique_ptr<Field>`

Query Interpretation



Operator Framework



Tuple Deserialization



Polymorphism
(Operator* rootOp)



Compute Overhead

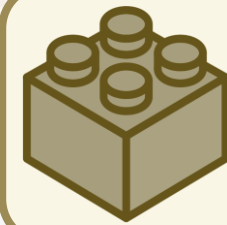
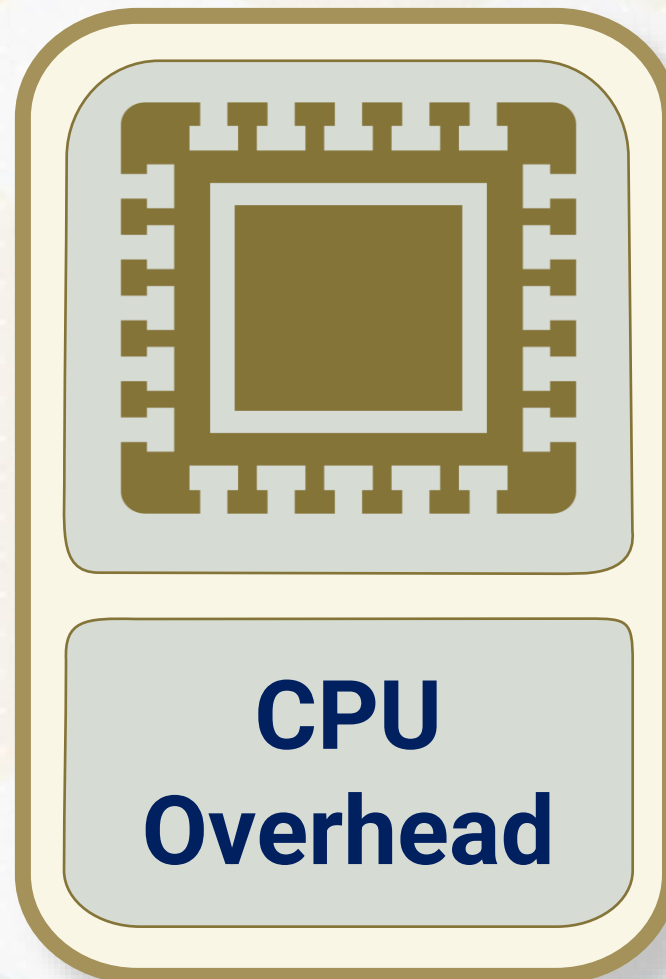


Memory Overhead



Temporary Objects
`std::unique_ptr<Field>`

Query Interpretation



Operator Framework



Tuple Deserialization



Polymorphism
(Operator* rootOp)



Compute Overhead

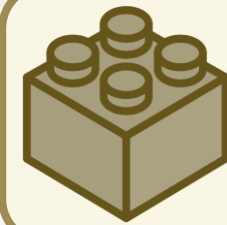
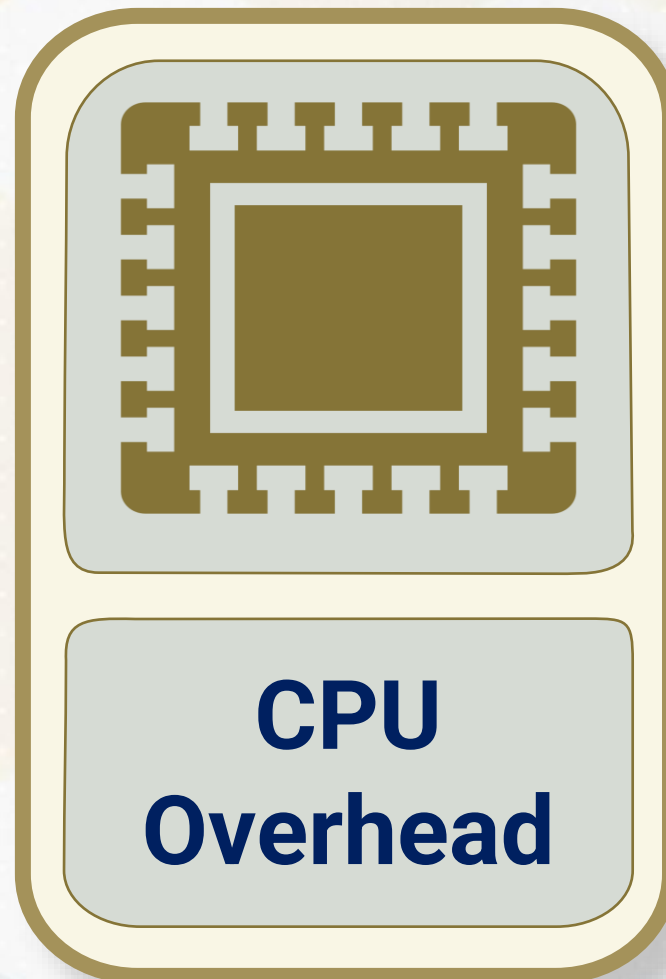


Memory Overhead



Temporary Objects
`std::unique_ptr<Field>`

Query Interpretation



Operator Framework



Tuple Deserialization



Polymorphism
(Operator* rootOp)



Compute Overhead

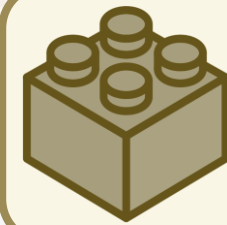
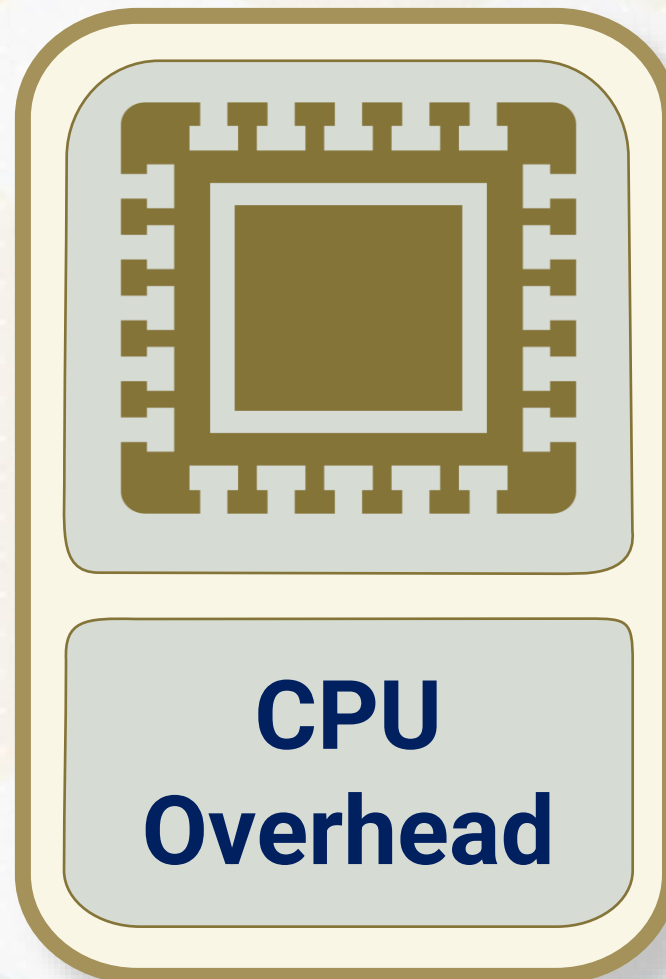


Memory Overhead



Temporary Objects
`std::unique_ptr<Field>`

Query Interpretation



Operator Framework



Tuple Deserialization



Polymorphism
(Operator* rootOp)



Compute Overhead

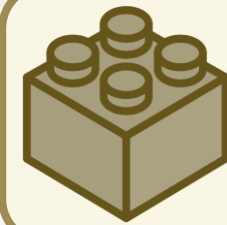
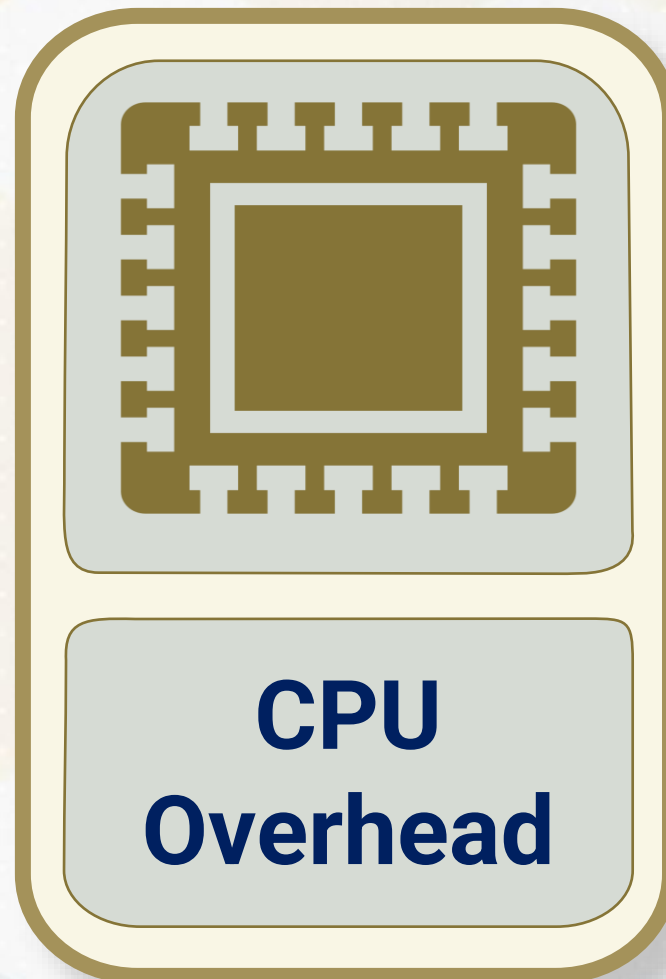


Memory Overhead



Temporary Objects
`std::unique_ptr<Field>`

Query Interpretation



Operator Framework



Tuple Deserialization



Polymorphism
(Operator* rootOp)



Compute Overhead



Memory Overhead



Temporary Objects
`std::unique_ptr<Field>`

Query Compilation

**SQL QUERY
STRING**

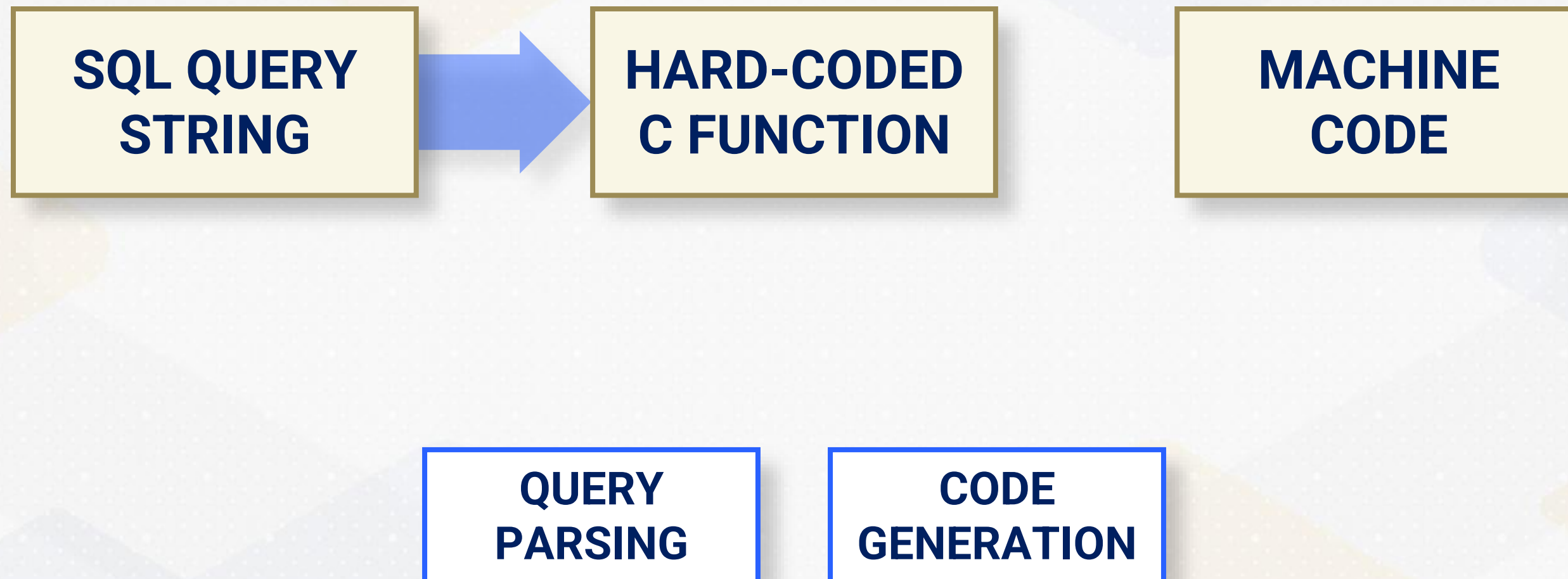
**HARD-CODED
C FUNCTION**

**MACHINE
CODE**

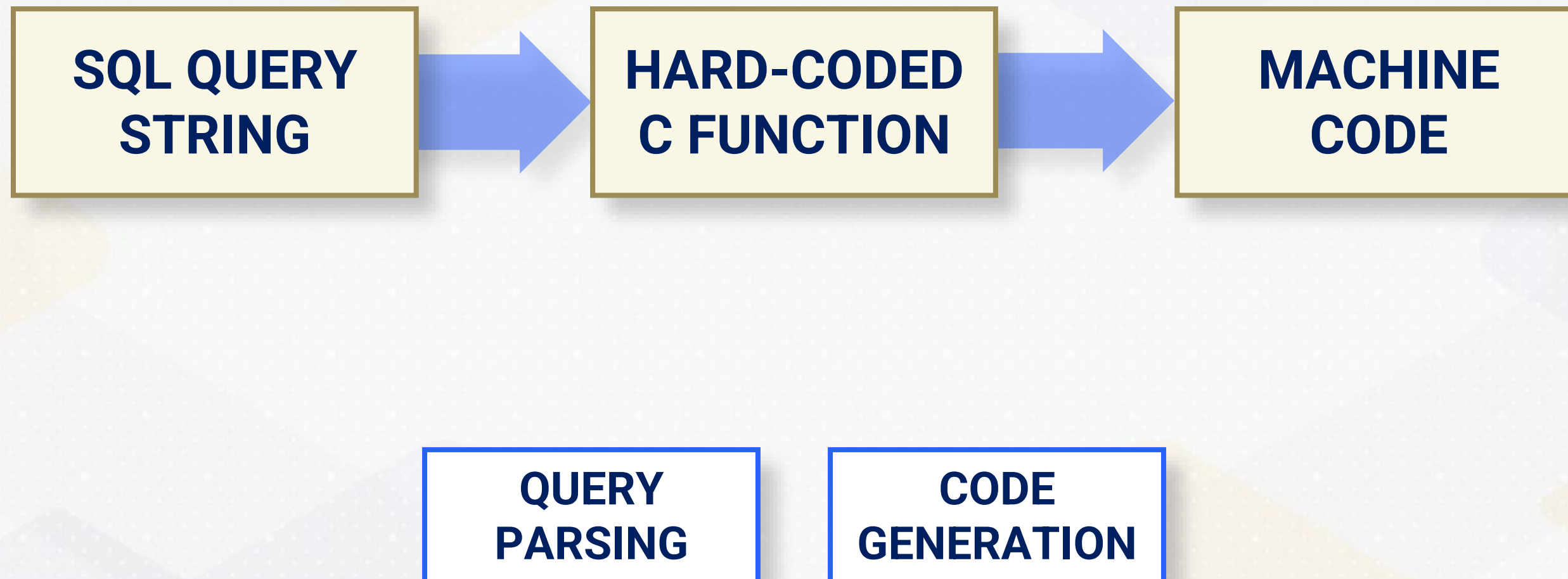
**QUERY
PARSING**

**CODE
GENERATION**

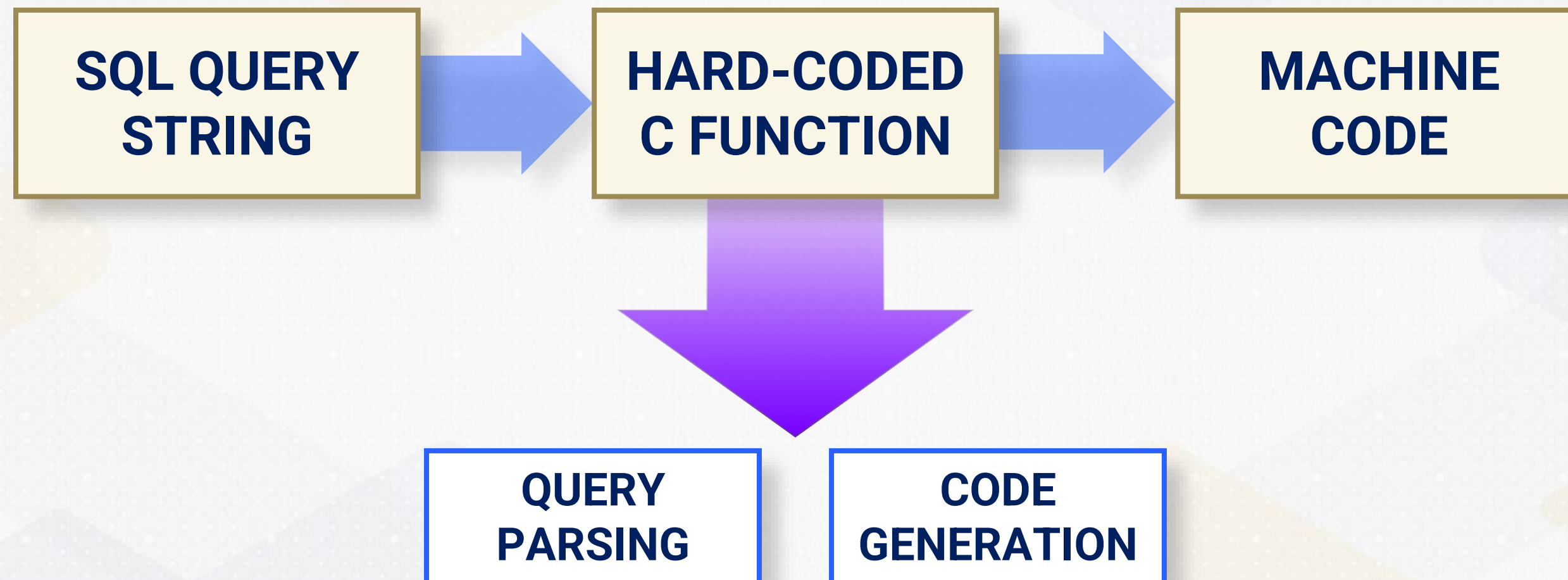
Query Compilation



Query Compilation



Query Compilation



Query Compilation: Scanning

```
void queryCompilation() {  
    while (currentPageIndex < buffer_manager.getNumPages()) {  
        auto &currentPage = buffer_manager.getPage(currentPageIndex);  
        Slot *slot_array = reinterpret_cast<Slot *>(page_buffer);  
        while (currentSlotIndex < MAX_SLOTS) {  
            if (!slot_array[currentSlotIndex].empty) {  
                ...           // Process tuple in slot  
                currentSlotIndex++; // Move to the next slot  
                continue;        // Continue scanning  
            }  
            currentSlotIndex++;  
        }  
    }  
}
```


Query Compilation: Scanning

queryCompilation

```
void queryCompilation() {  
    while (currentPageIndex < buffer_manager.getNumPages()) {  
        auto &currentPage = buffer_manager.getPage(currentPageIndex);  
        Slot *slot_array = reinterpret_cast<Slot *>(page_buffer);  
        while (currentSlotIndex < MAX_SLOTS) {  
            if (!slot_array[currentSlotIndex].empty) {  
                ...           // Process tuple in slot  
                currentSlotIndex++; // Move to the next slot  
                continue;         // Continue scanning  
            }  
            currentSlotIndex++;  
        }  
    }  
}
```

Query Compilation: Scanning



queryCompilation

Runs Operations

```
void queryCompilation() {  
    while (currentPageIndex < buffer_manager.getNumPages()) {  
        auto &currentPage = buffer_manager.getPage(currentPageIndex);  
        Slot *slot_array = reinterpret_cast<Slot *>(page_buffer);  
        while (currentSlotIndex < MAX_SLOTS) {  
            if (!slot_array[currentSlotIndex].empty) {  
                ...           // Process tuple in slot  
                currentSlotIndex++; // Move to the next slot  
                continue;        // Continue scanning  
            }  
            currentSlotIndex++;  
        }  
    }  
}
```


Query Compilation: Scanning

queryCompilation

Runs Operations

Direct Navigation

```
void queryCompilation() {  
    while (currentPageIndex < buffer_manager.getNumPages()) {  
        auto &currentPage = buffer_manager.getPage(currentPageIndex);  
        Slot *slot_array = reinterpret_cast<Slot *>(page_buffer);  
        while (currentSlotIndex < MAX_SLOTS) {  
            if (!slot_array[currentSlotIndex].empty) {  
                ...           // Process tuple in slot  
                currentSlotIndex++; // Move to the next slot  
                continue;        // Continue scanning  
            }  
            currentSlotIndex++;  
        }  
    }  
}
```

Query Compilation: Field Extraction

```
const char *tuple_data = page_buffer + slot_array[currentSlotIndex].offset;
// Navigate to the first field -- text based hard-coded deserialization
// "4 0 4 2 0 4 231 1 4 132.04 2 7 buzzdb"
std::string integerField;
int character_count = 0; int current_count = 0;
int required_count = 4; // to retrieve 2 (first field)
while (current_count < required_count) {
    integerField = "";
    for (const char *p = tuple_data + character_count; *p != ' ' && *p != '\0'; ++p) {
        character_count++; integerField += *p;
    }
    character_count++; current_count++;
}
int fieldValue = std::stoi(integerField);
```


Query Compilation: Field Extraction

```
const char *tuple_data = page_buffer + slot_array[currentSlotIndex].offset;
// Navigate to the first field -- text based hard-coded deserialization
// "4 0 4 2 0 4 231 1 4 132.04 2 7 buzzdb"
std::string integerField;
int character_count = 0; int current_count = 0;
int required_count = 4; // to retrieve 2 (first field)
while (current_count < required_count) {
    integerField = "";
    for (const char *p = tuple_data + character_count; *p != ' ' && *p != '\0'; ++p) {
        character_count++; integerField += *p;
    }
    character_count++; current_count++;
}
int fieldValue = std::stoi(integerField);
```

Query Compilation: Aggregation

```
// Maps to store the sum and count for each group
std::unordered_map<int, int> groupSums;

// Apply WHERE condition
if (fieldValue > 2 && fieldValue < 6) {
    groupSums[fieldValue] += fieldValue;
}

// Output the results
for (const auto &group : groupSums) {
    std::cout << "KEY " << group.first;
    std::cout << ", SUM: " << groupSums[group.first] << std::endl;
}
```


Advantages of Query Compilation



Advantages of Query Compilation



1

REDUCED INTERPRETATION OVERHEAD

Advantages of Query Compilation



1

REDUCED INTERPRETATION OVERHEAD

2

INCREASED EXECUTION SPEED

Advantages of Query Compilation



1

REDUCED INTERPRETATION OVERHEAD

COMPILED QUERY 30 times faster than INTERPRETED QUERY

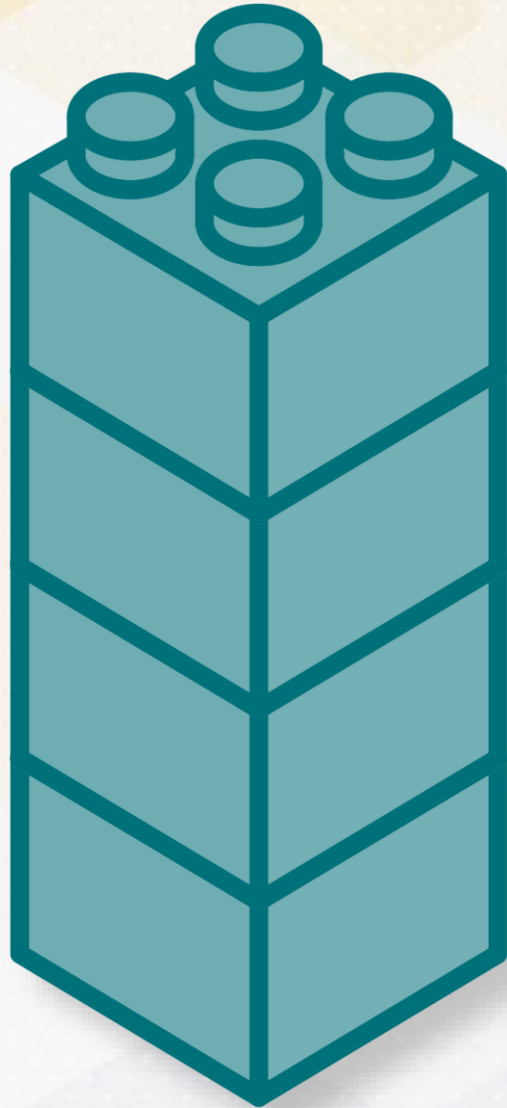
2

INCREASED EXECUTION SPEED

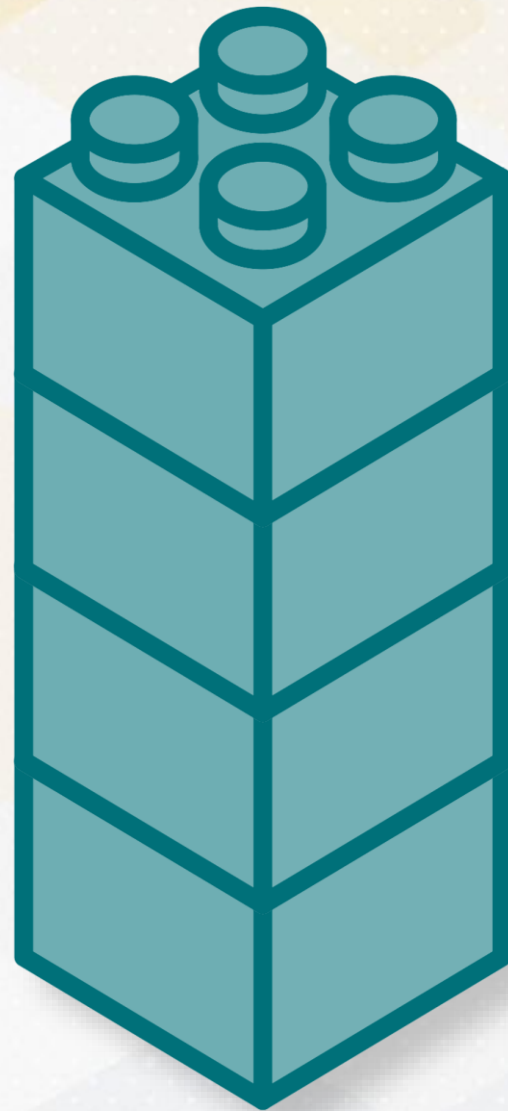
Drawbacks of Query Compilation



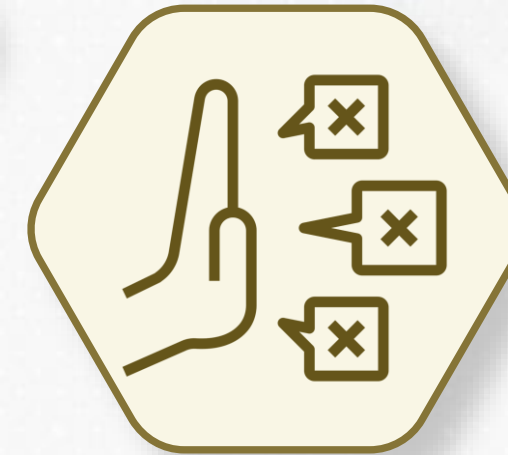
Drawbacks of Query Compilation



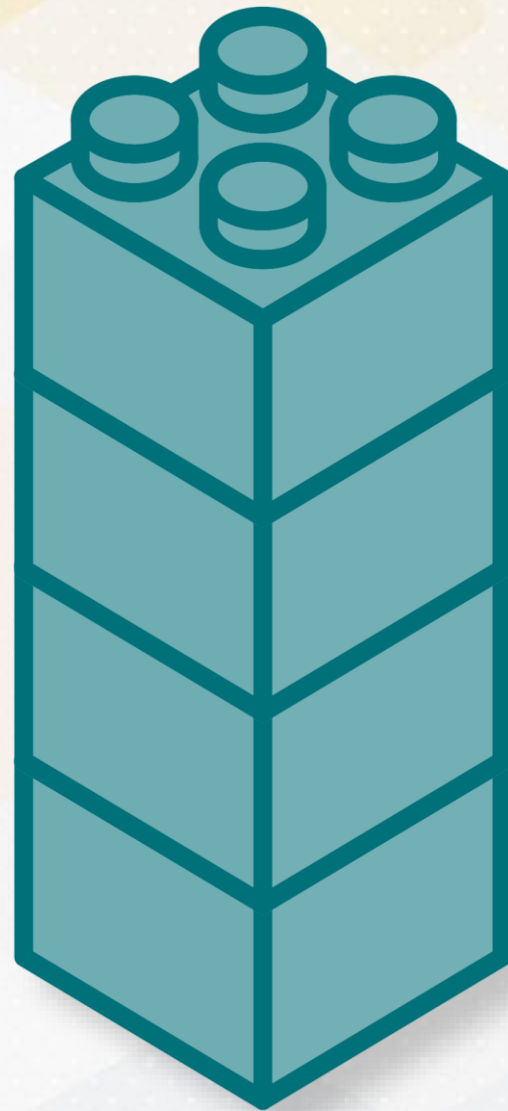
Drawbacks of Query Compilation



Compilation
Process



Drawbacks of Query Compilation



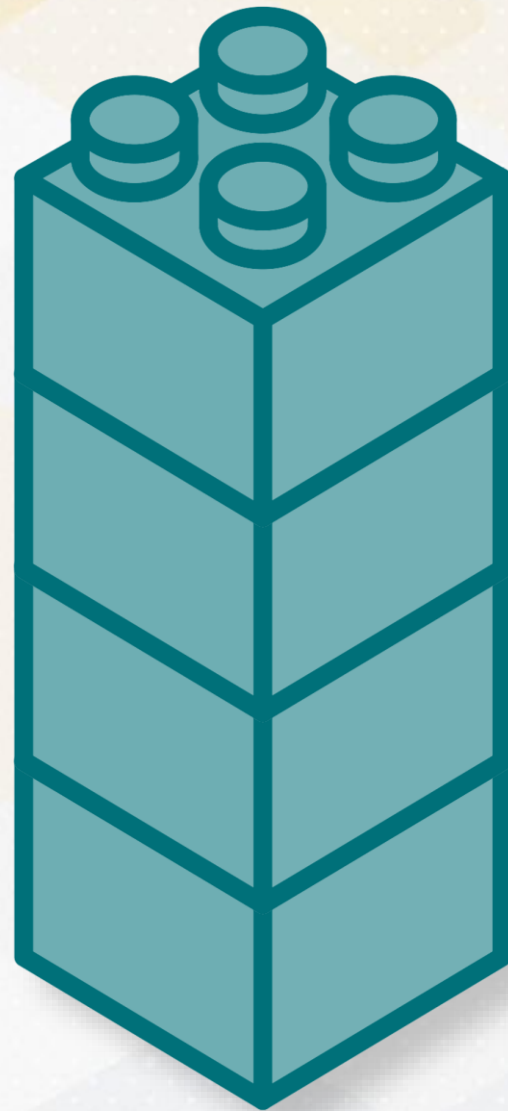
Compilation
Process



Long
Compile
Time



Drawbacks of Query Compilation



Compilation
Process



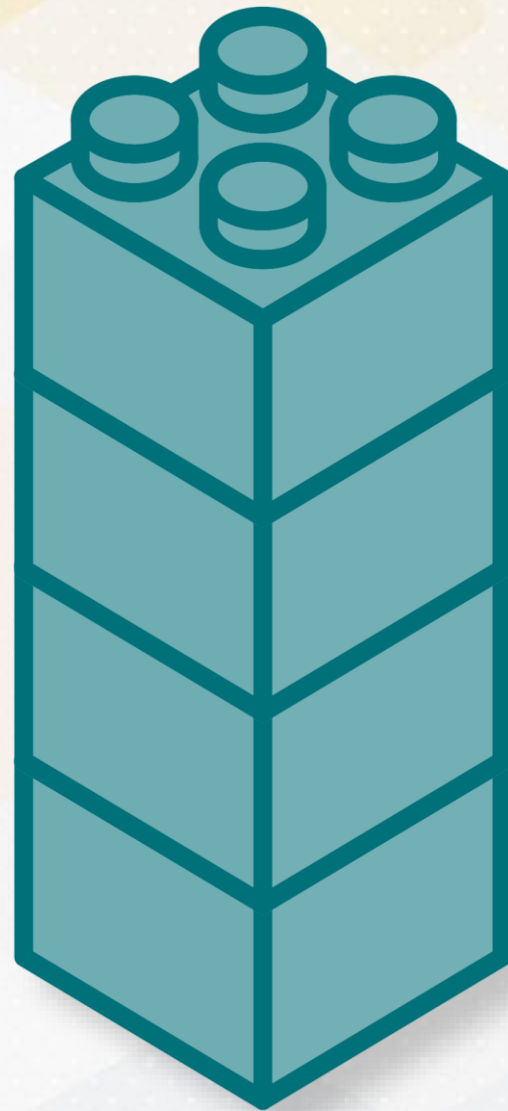
Long
Compile
Time



Reduced
Flexibility



Drawbacks of Query Compilation



Compilation
Process



Long
Compile
Time



Reduced
Flexibility



Additional
Compile
Time





Query Compilation



**Query
Compilation**

**Query
Interpretation**